

Activity Selection Problem

1. What is the problem asking?

You are given several activities.

For every activity:

- **start[i]** = when it starts
- **finish[i]** = when it ends

You have **only one person**, so they can do only one activity at a time.

The goal is:

Select the maximum number of activities such that no two selected activities overlap.

Important condition

If one activity finishes at time x , the next activity must start **strictly greater than** x .

So:

```
previous finish < next start
```

⚠ Notice it is **not** $\leq$.

For example:

```
Activity A: [1, 3]
Activity B: [3, 5]
```

These **cannot** both be selected because:

```
3 < 3 ❌
```

2. Key observation

The main question is:

Which activity should we choose first?

Suppose we have:

```
A: [1, 10]
B: [2, 3]
C: [4, 5]
```

If we choose **A** first:

```
[1-----10]
```

we cannot choose anything afterward.

But if we choose **B**:

```
[2--3]
```

we can then choose **C**:

```
[2--3]    [4--5]
```

So we should prefer activities that **finish earlier**.

Greedy idea

Always choose the activity that finishes earliest among the activities we can currently choose.

This is the famous **Activity Selection Greedy Algorithm**.

3. Build the intuition step-by-step

Step 1: Brute force

We could try every possible subset of activities.

For each activity, we have two choices:

```
Take it
OR
Don't take it
```

For n activities, that gives roughly:

```
2^n
```

possibilities.

For $n = 2 * 10^5$, this is obviously impossible.

Step 2: Can sorting help?

Yes.

Suppose we sort activities by their **finish time**.

Example:

Activity	Start	Finish
A	1	2
B	3	4
C	0	6
D	5	7
E	8	9
F	5	9

Already sorted by finish time:

```
A → B → C → D → E → F
2   4   6   7   9   9
```

Now we greedily process them from left to right.

Step 3: Why earliest finish?

Imagine we have two possible activities:

```
A: [1, 4]
B: [2, 3]
```

Both can be selected now.

Which one is better?

```
A finishes at 4
B finishes at 3
```

Choosing **B** leaves us more time for future activities.

```
B: [2---3]
    ↑
  free earlier
```

Choosing the activity that finishes earliest can **never hurt us**, because it leaves at least as much room for everything that comes afterward.

That's the greedy-choice intuition.

4. The greedy algorithm

We maintain:

```
C++
lastFinish
```

which tells us when our previously selected activity finished.

Initially:

```
C++
lastFinish = -1;
```

Then:

1. Sort all activities by **finish**.
2. Look at activities in that order.
3. If:

```
C++
start > lastFinish
```

then we can select the activity.

4. Update:

```
C++
lastFinish = finish
```

5. Increase our answer.

5. Example

Input:

```
start = [1, 3, 0, 5, 8, 5]
finish = [2, 4, 6, 7, 9, 9]
```

Create pairs:

```
(start, finish)
(1,2)
(3,4)
(0,6)
(5,7)
(8,9)
(5,9)
```

Sort by finish:

```
(1,2)
(3,4)
(0,6)
(5,7)
(8,9)
(5,9)
```

Now process them.

Activity (1,2)

Initially:

```
lastFinish = -1
```

Check:

```
1 > -1 ✓
```

Take it.

```
answer = 1
lastFinish = 2
```

Activity (3,4)

```
3 > 2 ✓
```

Take it.

```
answer = 2
lastFinish = 4
```

Activity (0,6)

```
0 > 4 ✗
```

Cannot take it.

Activity (5,7)

5 > 4

Take it.

```
answer = 3
lastFinish = 7
```

Activity (8,9)

8 > 7

Take it.

```
answer = 4
lastFinish = 9
```

Activity (5,9)

5 > 9

Cannot take it.

Final answer:

4

Selected activities:

```
(1,2)
(3,4)
(5,7)
(8,9)
```

Visual representation:

```
Time:
1---2   3---4   5-----7   8---9
| A |   | B |   | D   |   | E |
```

No activities overlap.

6. Why does this greedy strategy work?

Suppose the earliest finishing activity is:

A finishes at 3

and another possible activity finishes at:

B finishes at 5

If we choose A, we become free at time 3.

If we choose B, we become free only at time 5.

Therefore:

```
Choosing A:
free from 3 onward
      ↓
----3----->

Choosing B:
free from 5 onward
      ↓
----3---5----->
```

Everything that could be done after B can also potentially be done after A.

So choosing the earliest finishing activity gives us the **maximum remaining room** for future activities.

That's why the greedy choice works.

7. C++ Code

```
C++
class Solution {
public:
    int activitySelection(vector<int>& start, vector<int>& finish) {
        int n = start.size();
        // Store {finish, start}
        vector<pair<int, int>> activities;

        for (int i = 0; i < n; i++) {
            activities.push_back({finish[i], start[i]});
        }

        // Sort by finish time
        sort(activities.begin(), activities.end());

        int count = 0;
        int lastFinish = -1;

        for (auto& activity : activities) {
            int finishTime = activity.first;
            int startTime = activity.second;

            // Next activity must start strictly after
            // previous activity finishes
            if (startTime > lastFinish) {
                count++;
                lastFinish = finishTime;
            }
        }

        return count;
    }
};
```

8. How the code works

Step 1: Store activities

We store:

```
C++
{finish[i], start[i]}
```

instead of:

```
C++
{start[i], finish[i]}
```

because we want sorting to happen primarily according to **finish time**.

For example:

```
start = [1, 3, 0]
finish = [2, 4, 6]
```

becomes:

```
{2,1}
{4,3}
{6,0}
```

Step 2: Sort

```
C++
sort(activities.begin(), activities.end());
```

Since **finish** is the first element of the pair, C++ sorts according to finish time.

Step 3: Track previous finish

```
C++
int lastFinish = -1;
```

This means:

"The last selected activity currently finishes at **lastFinish**."

Step 4: Check compatibility

```
C++
if (startTime > lastFinish)
```

This is the most important line.

We need:

```
startTime > previousFinish
```

not:

```
startTime >= previousFinish
```

because the problem explicitly says the next activity must start **after** the previous activity finishes.

Step 5: Select it

```
C++
count++;
lastFinish = finishTime;
```

Once we select the activity:

- increase answer
- make its finish time the new boundary

9. Complete dry run

Input:

```
start = [1, 3, 0, 5, 8, 5]
finish = [2, 4, 6, 7, 9, 9]
```

Activities:

Activity	Start	Finish
0	1	2
1	3	4
2	0	6
3	5	7
4	8	9
5	5	9

After sorting by finish:

Order	Start	Finish
1	1	2
2	3	4
3	0	6
4	5	7
5	8	9
6	5	9

Initially:

```
count = 0
lastFinish = -1
```

Iteration 1

```
start = 1
finish = 2
```

Check:

```
1 > -1 
```

Select.

```
count = 1
lastFinish = 2
```

Iteration 2

```
start = 3
finish = 4
```

Check:

```
3 > 2 
```

Select.

```
count = 2
lastFinish = 4
```

Iteration 3

```
start = 0
finish = 6
```

Check:

```
0 > 4 
```

Skip.

```
count = 2
lastFinish = 4
```

Iteration 4

```
start = 5
finish = 7
```

Check:

```
5 > 4 
```

Select.

```
count = 3
lastFinish = 7
```

Iteration 5

```
start = 8
finish = 9
```

Check:

```
8 > 7 
```

Select.

```
count = 4
lastFinish = 9
```

Iteration 6

```
start = 5
finish = 9
```

Check:

```
5 > 9 
```

Skip.

Final:

```
count = 4
```

Selected activities

```
Activity 0 → (1,2)
Activity 1 → (3,4)
Activity 3 → (5,7)
Activity 4 → (8,9)
```

Therefore:

```
Answer = 4
```

10. Important edge case

Consider:

```
start = [10, 12, 20]
finish = [20, 25, 30]
```

Activities:

```
(10, 20)
(12, 25)
(20, 30)
```

We choose:

```
(10, 20)
```

Now:

```
lastFinish = 20
```

For (20,30):

```
20 > 20 ❌
```

So we **cannot** choose it.

Answer:

```
1
```

This is why the condition must be:

```
C++
startTime > lastFinish
```

11. Complexity

Let n be the number of activities.

Sorting

```
C++
sort(...)
```

takes:

```
 $O(n \log n)$ 
```

Traversal

We check every activity once:

```
 $O(n)$ 
```

Therefore total:

```
 $O(n \log n)$ 
```

Space

We store n pairs:

```
 $O(n)$ 
```


So:

Time Complexity: $O(n \log n)$
Space Complexity: $O(n)$

The pattern to remember

Whenever you see:

"Select maximum number of non-overlapping intervals/activities"

think:

1. Create (start, finish)
2. Sort by FINISH time
3. Pick the earliest finishing compatible activity
4. Update lastFinish
5. Repeat

The core greedy idea is simply:

Finish early → become free early → get more chances to select future activities.

ChatGPT can make mistakes. Check important info.